

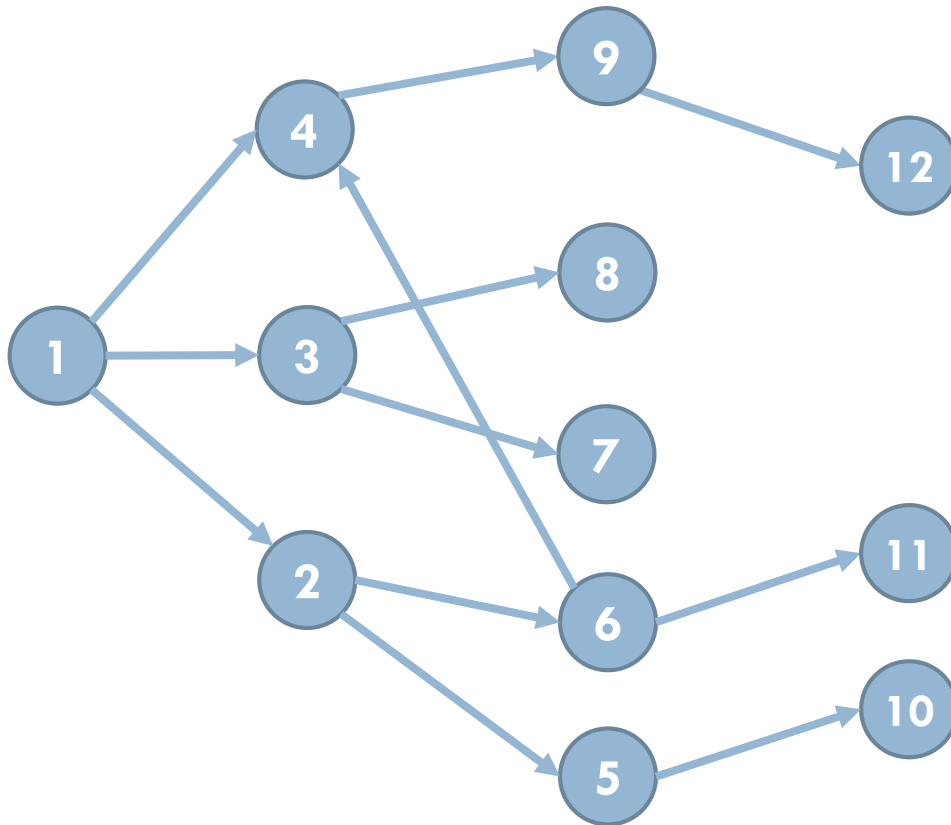
GRAF

REKUZIVNI OBILAZAK PO
DUBINI
DIJKSTRA-IN ALGORITAM

Rekurzivni obilazak grafa po dubini

- Odrediti broj čvorova grafa koji se mogu obići iz zadatog čvora
- Koristiti obilazak grafa po dubini
- Obilazak implementirati rekurzivno

Obilazak grafa



□ Obilazak po širini

□ 1 2 3 4 5 6 7 8
9 10 11 12

□ Obilazak po dubini (stek)

□ 1 2 5 10 6 11 3
7 8 4 9 12

□ Obilazak po dubini (rekurzija)

□ 1 2 5 10 6 4 9 12
11 3 7 8

Rekurzivni obilazak grafa po dubini

```
template <class T, class W>
long GraphAsLists<T,W>::dfs(T first)
{
    LinkedNode<T,W>* pFirst = nullptr;
    LinkedNode<T,W> *ptr = start;
    while(ptr != nullptr){
        ptr->status = 0;
        if (ptr->node == first) {
            pFirst = ptr;
        }
        ptr = ptr->next;
    }
    if (pFirst == nullptr)
        return 0;
    return dfs(pFirst);
}
```

Rekurzivni obilazak grafa po dubini

```
template <class T, class W>
long GraphAsLists<T,W>::dfs(LinkedList<T,W>* ptr)
{
    int retVal = 0;
    if (ptr->status != 1) {
        ptr->status = 1;
        ptr->visit();
        retVal++;
        Edge<T,W>* pot = ptr->adj;
        while(pot != nullptr){
            retVal += dfs(pot->dest);
            pot = pot->link;
        }
    }
    return retVal;
}
```

Rekurzivno određivanje puta između dva čvora obilaskom po dubini

- Odrediti put između dva čvora u grafu
 - ▣ Put čine svi čvorovi kroz koje treba proći da bi se stiglo iz jednog u drugi čvor
- Koristiti obilazak grafa po dubini
- Obilazak implementirati rekurzivno

Rekurzivno određivanje puta između dva čvora obilaskom po dubini

```
template <class T, class W> long GraphAsLists<T,W>::findPath
    (T first, T last, T *arPath/*=nullptr*/, int
     *lPath/*=nullptr*/)
{
    LinkedNode<T,W> *pFirst = nullptr, *pLast = nullptr;
    LinkedNode<T,W> *ptr = start;
    while (ptr != nullptr) {
        ptr->status = 0;
        if (ptr->node == first)
            pFirst = ptr;
        if (ptr->node == last)
            pLast = ptr;
        ptr = ptr->next;
    }
    if (pFirst == nullptr || pLast == nullptr)
        return 0;
    return findPath(pFirst, pLast, arPath, lPath);
}
```

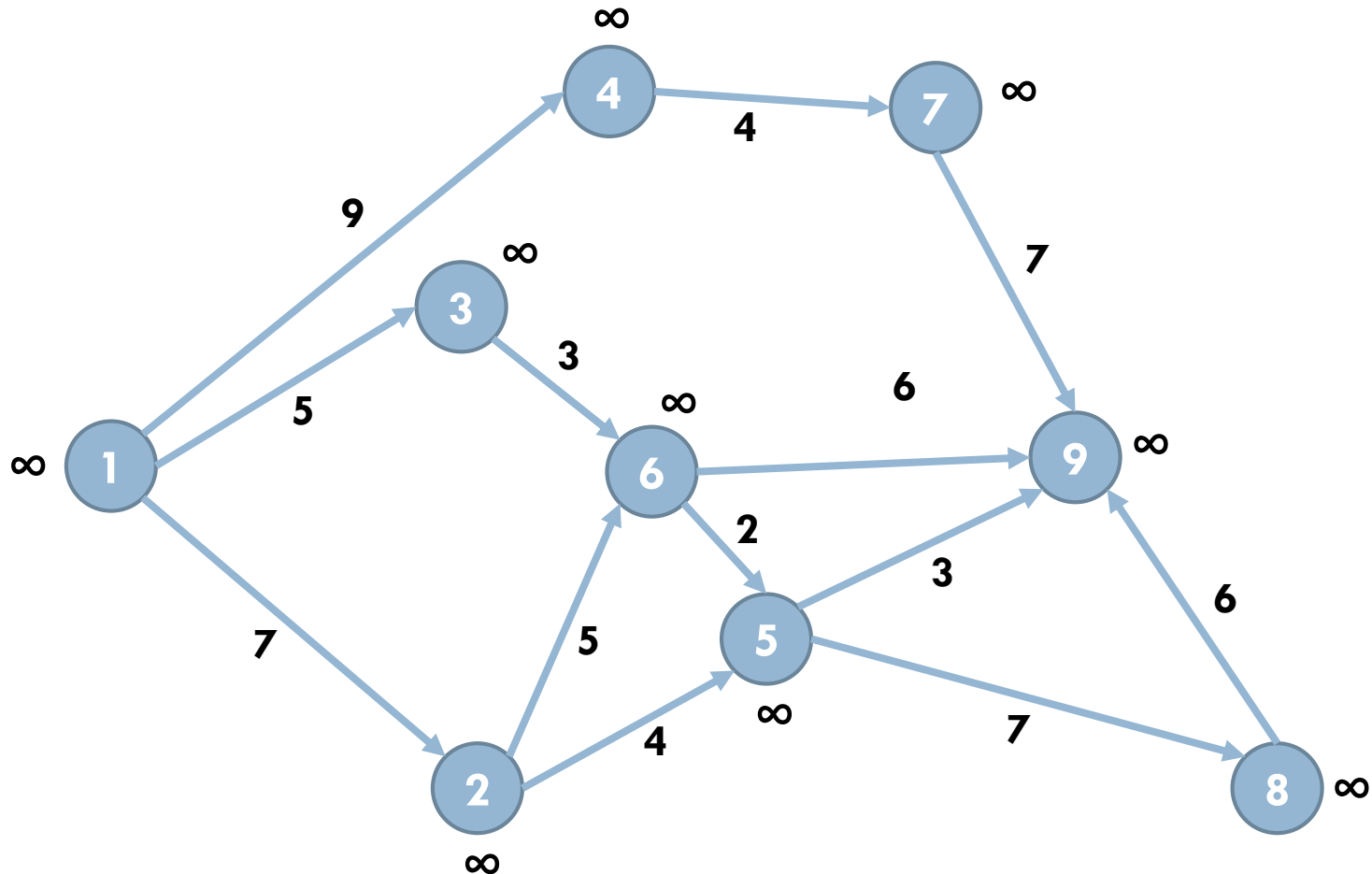
Rekurzivno određivanje puta između dva čvora obilaskom po dubini

```
template <class T, class W>
long GraphAsLists<T,W>::findPath(LinkedNode<T,W>* ptr,
    LinkedNode<T,W>* pEnd, T *arPath/*=nullptr*/, int *lPath/*=nullptr*/) {
    if (ptr->node == pEnd->node) {
        if (arPath != nullptr)
            arPath[(*lPath)++] = ptr->node;
        return 1;
    }
    int retVal = 0;
    if (ptr->status != 1) {
        ptr->status = 1;
        retVal++;
        Edge<T,W>* pot = ptr->adj;
        while (lPath != nullptr && *lPath == 0 && pot != nullptr) {
            retVal += findPath(pot->dest, pEnd, arPath, lPath);
            pot = pot->link;
        }
        if (lPath != nullptr && *lPath != 0 && arPath != nullptr)
            arPath[(*lPath)++] = ptr->node;
    }
    return retVal;
}
```

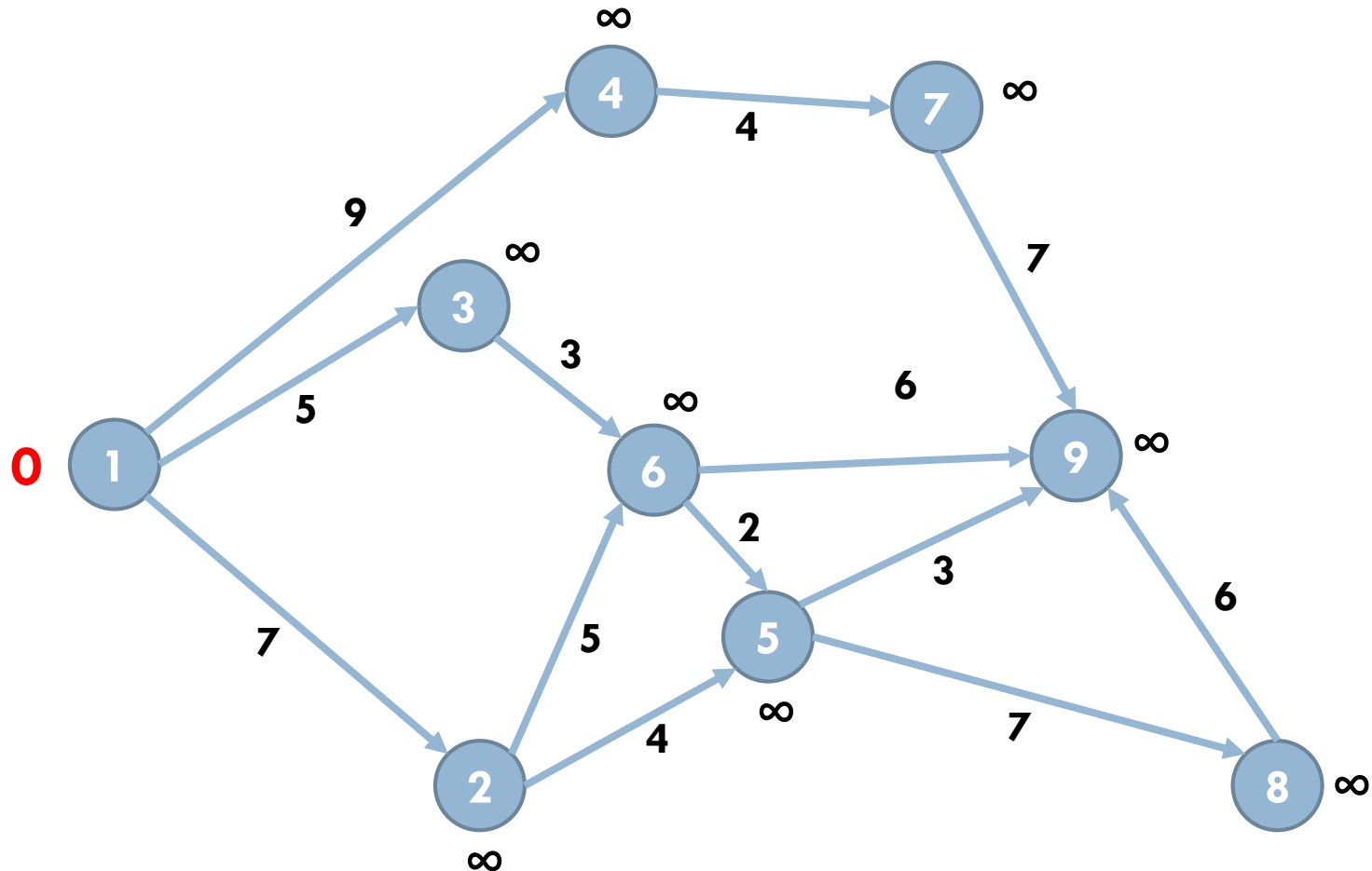
Najkraći put – Dijkstra

- svim čvorovima postaviti status *obrađen* na **false** i *rastojanje* na ∞
- izabrati startni čvor **s** i postaviti *rastojanje* na **0**
- dok nisu obrađeni svi čvorovi
 - ▣ za radni čvor **r** izabrati (od svih čvorova u grafu) čvor sa najmanjom vrednošću *rastojanja* čiji je status *obrađen* = **false** i promeniti ga u **true**
 - ▣ za sve susede **v_i** čvora **r** čiji je status *obrađen* = **false** proveriti da li je vrednost **v_i.rastojanja** > **r.rastojanje** + **C(r,v_i)** i ako jeste postaviti **v_i.rastojanja** = **r.rastojanje** + **C(r,v_i)**

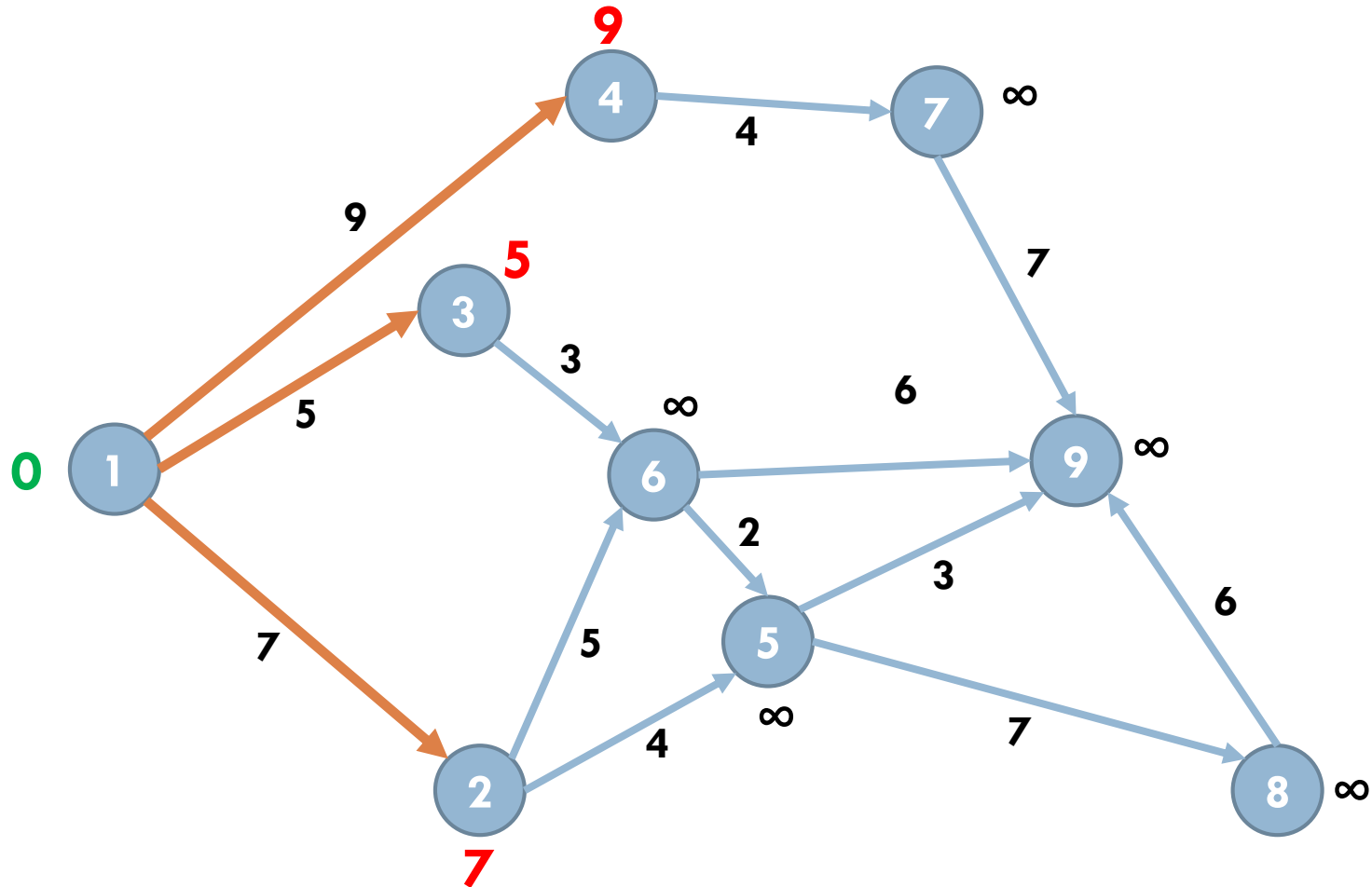
Najkraći put – Dijkstra



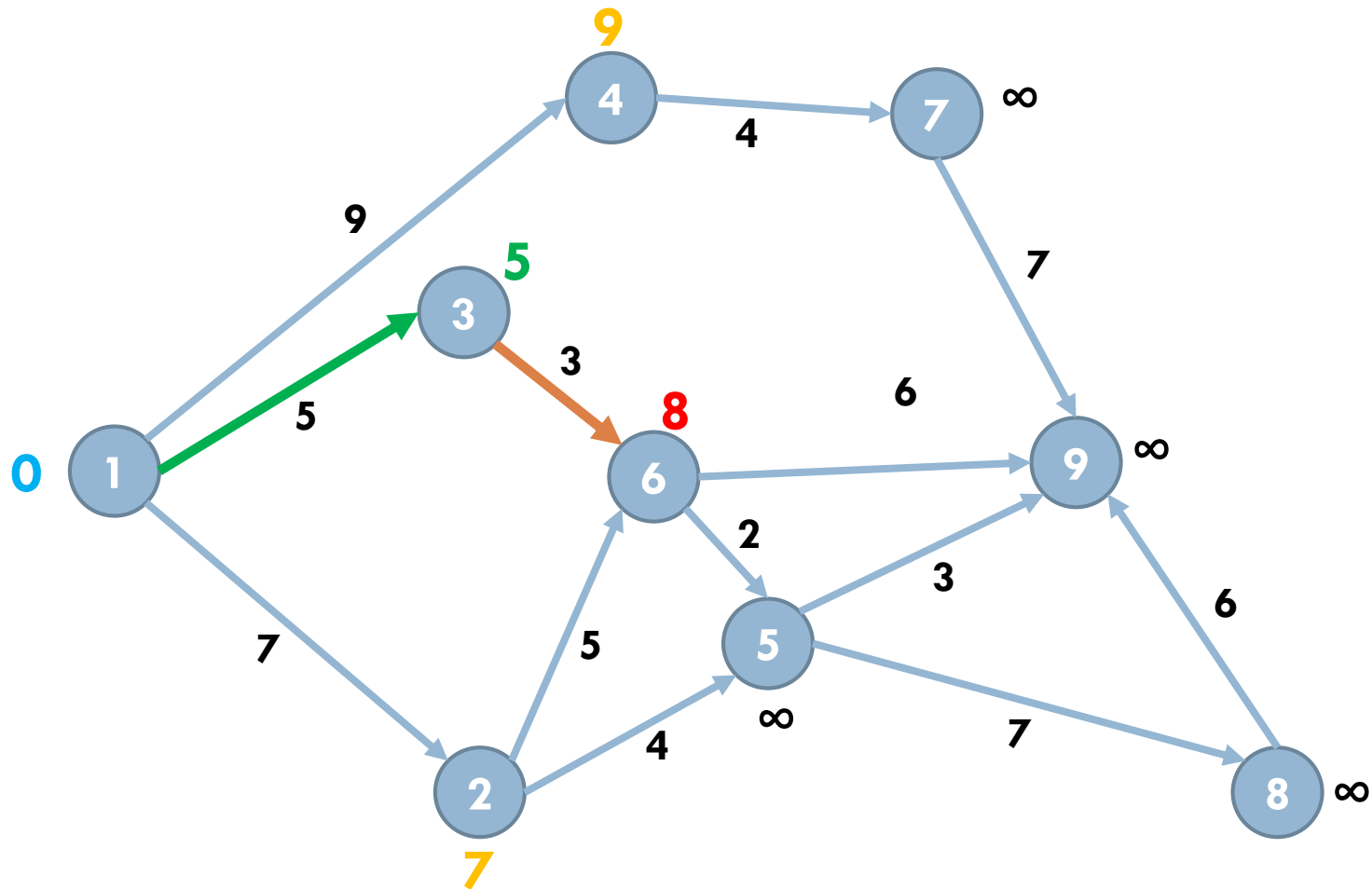
Najkraći put – Dijkstra



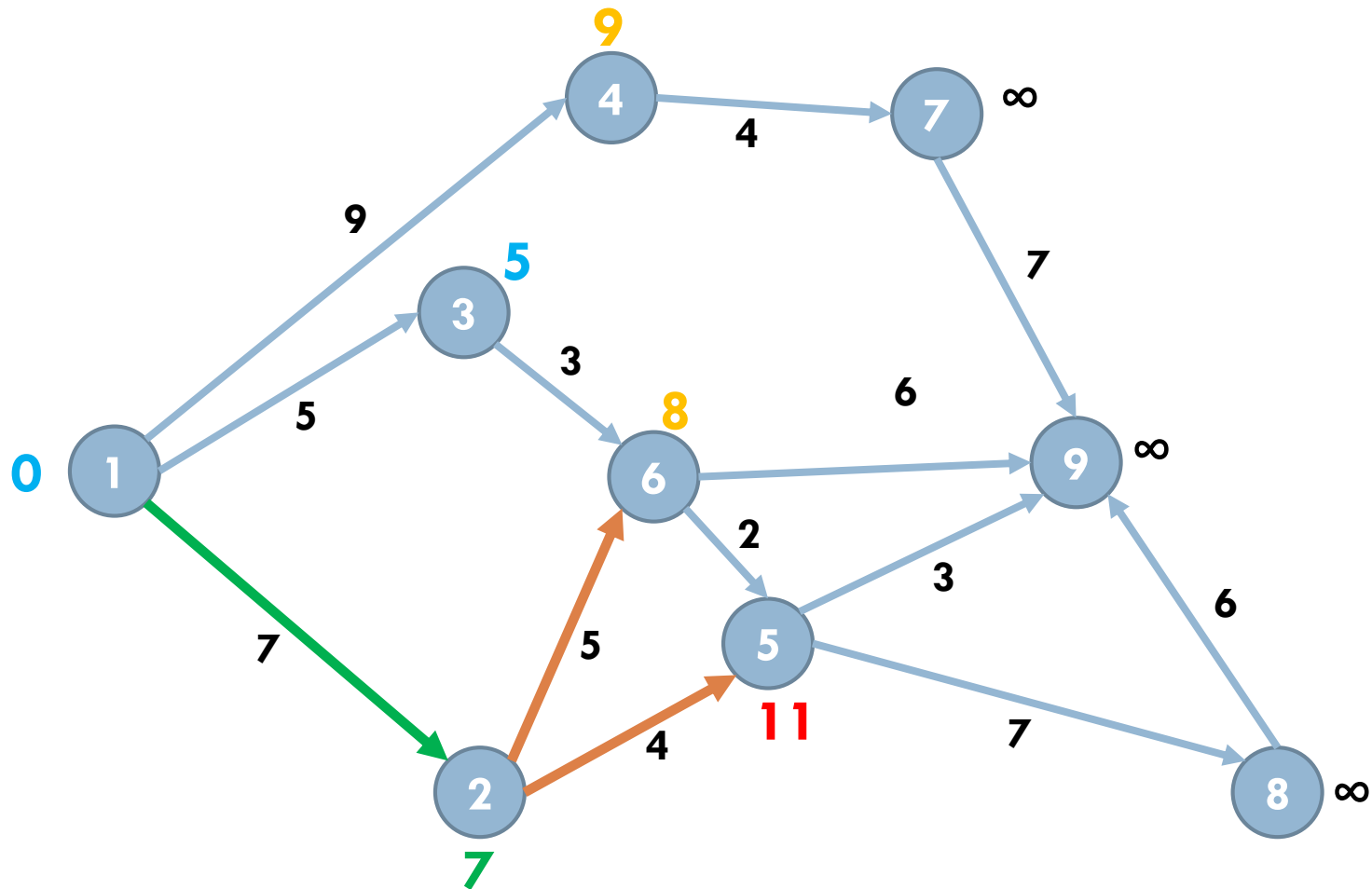
Najkraći put – Dijkstra



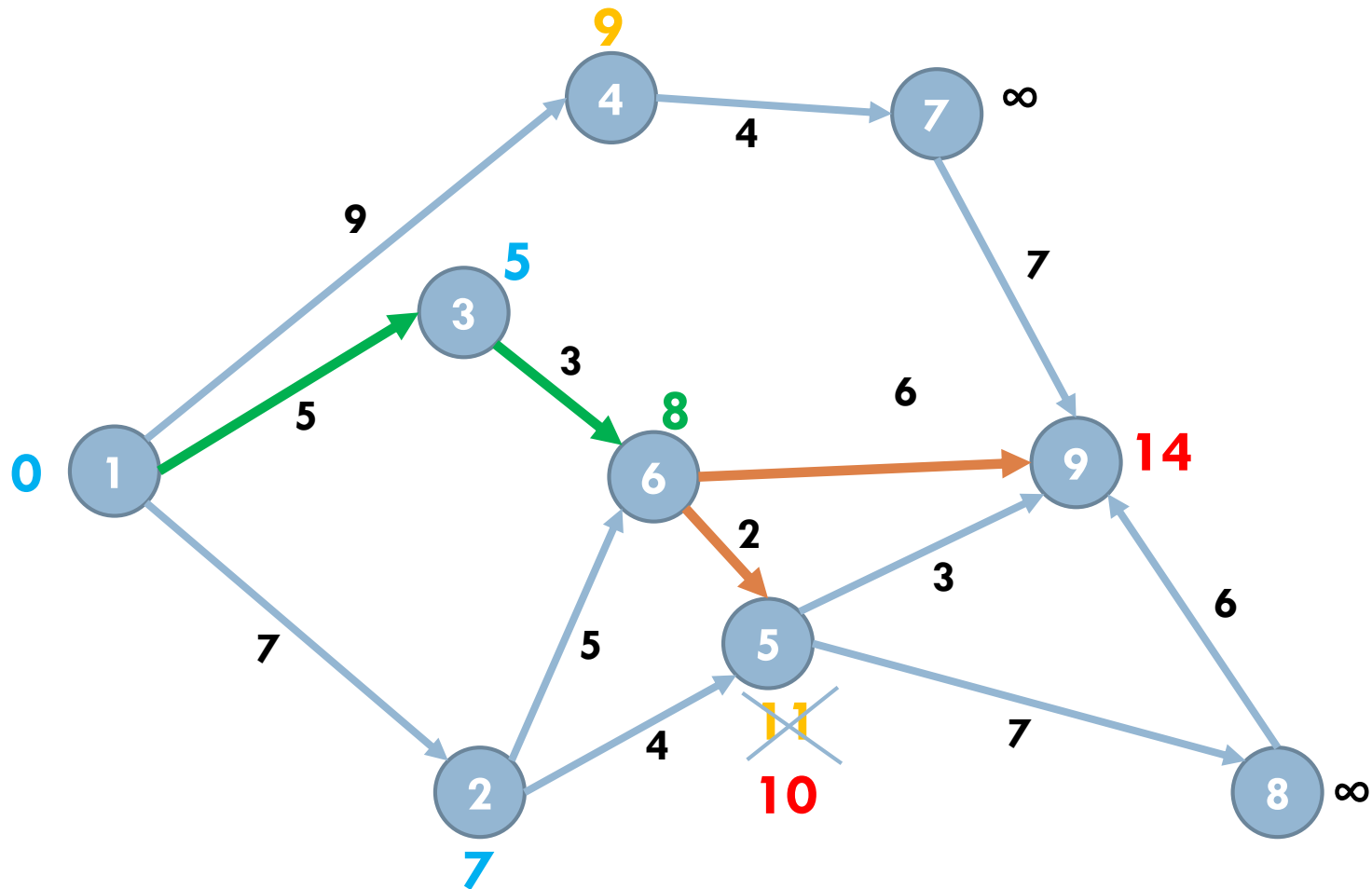
Najkraći put – Dijkstra



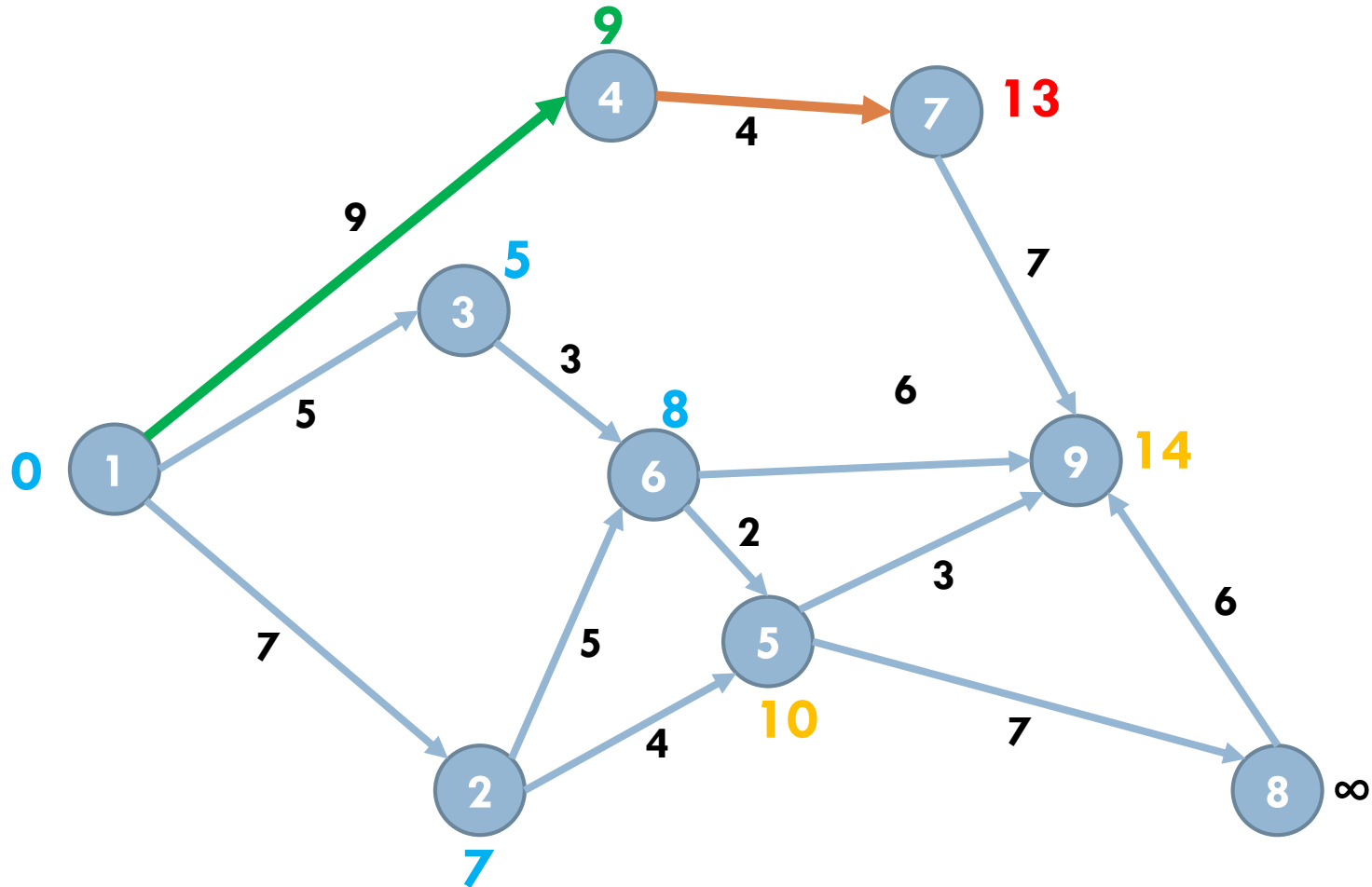
Najkraći put – Dijkstra



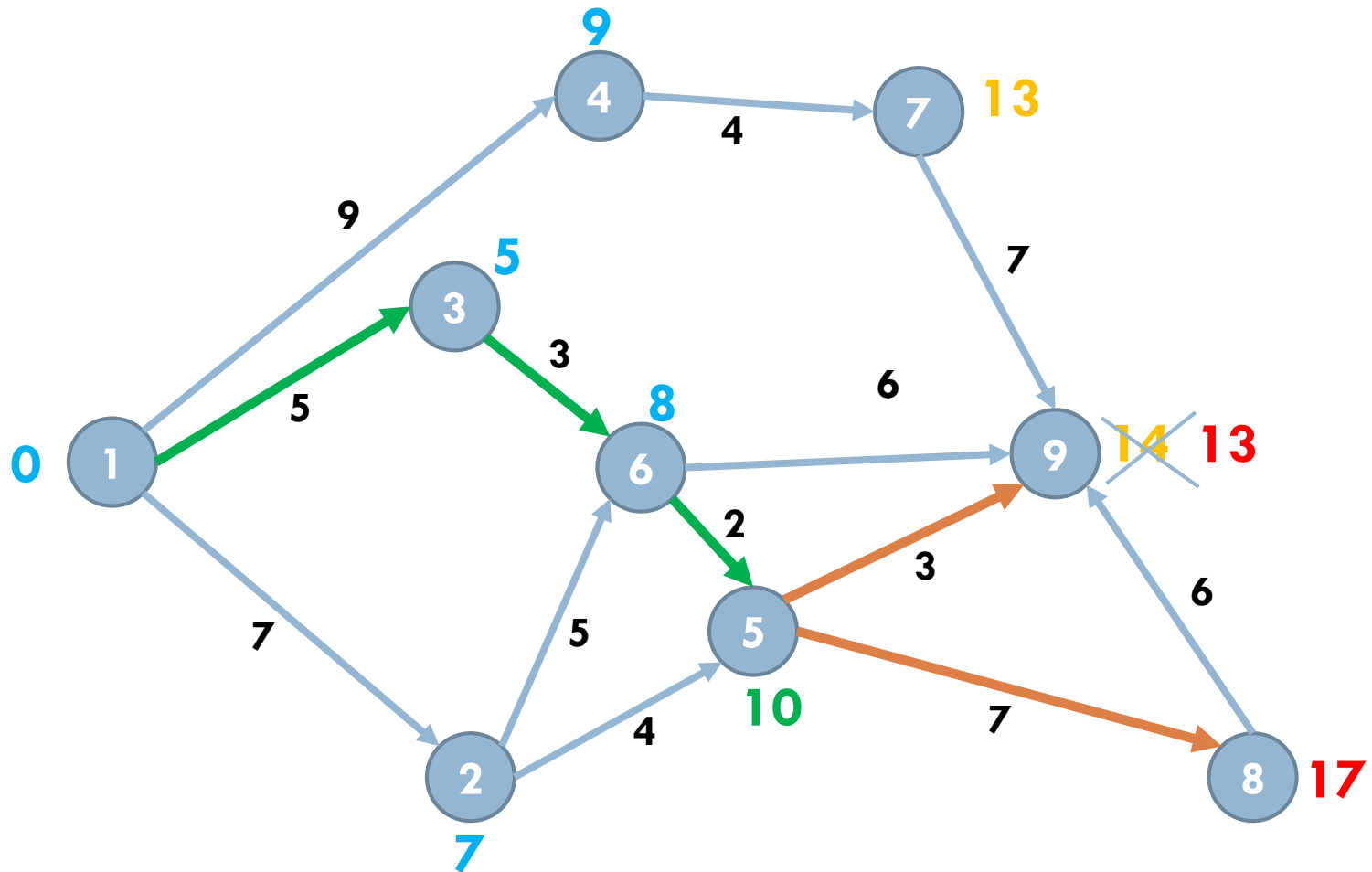
Najkraći put – Dijkstra



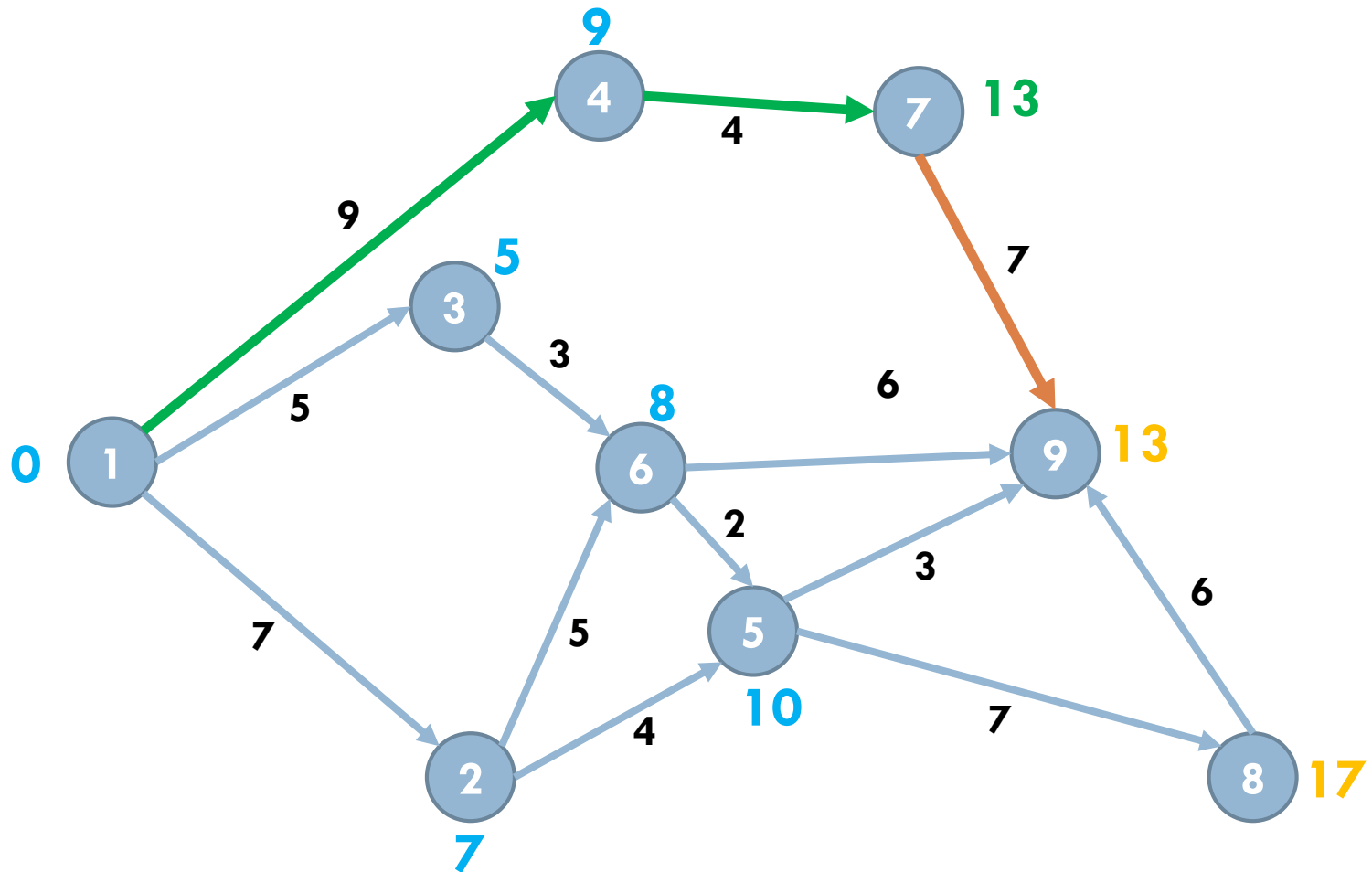
Najkraći put – Dijkstra



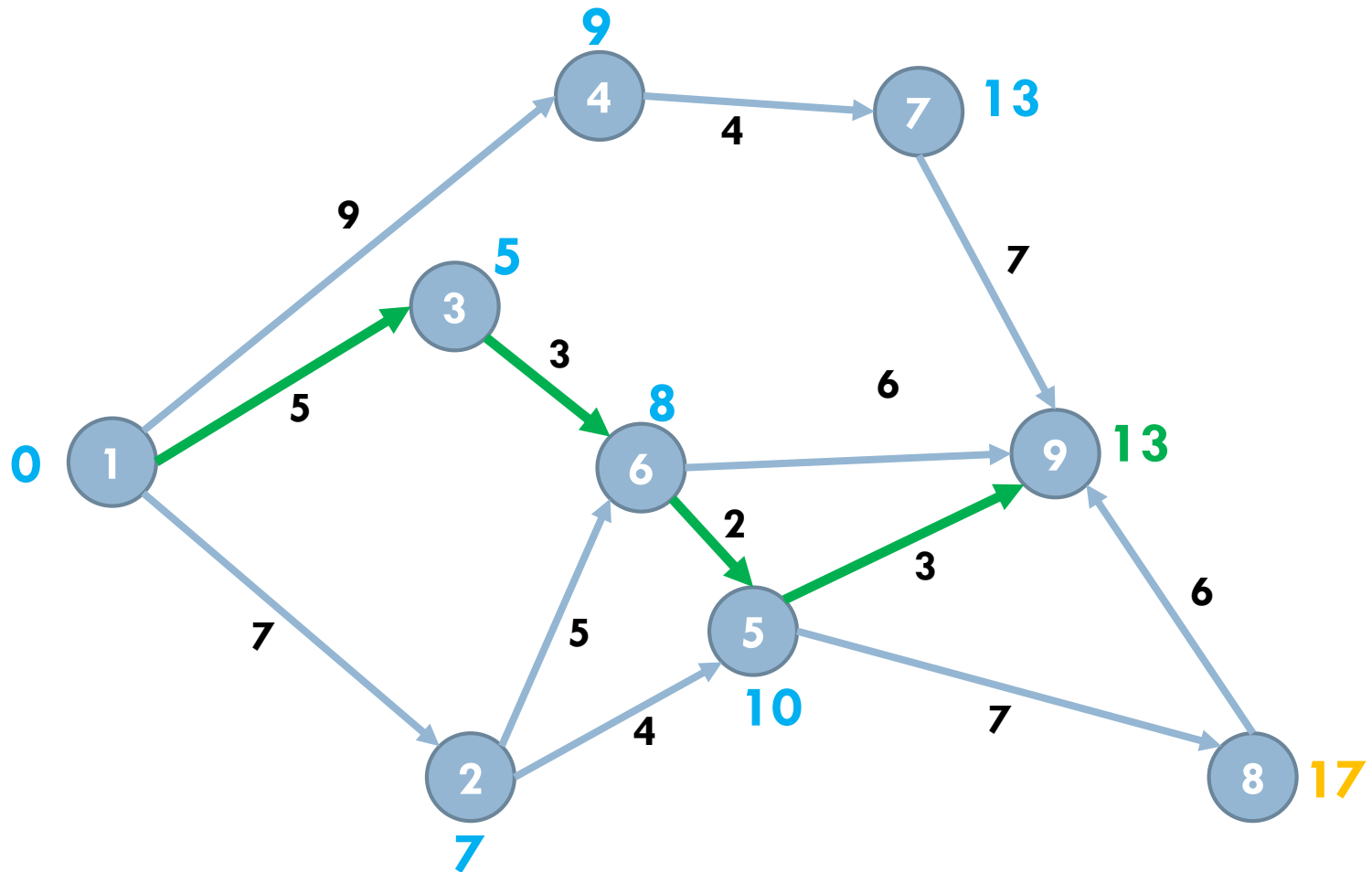
Najkraći put – Dijkstra



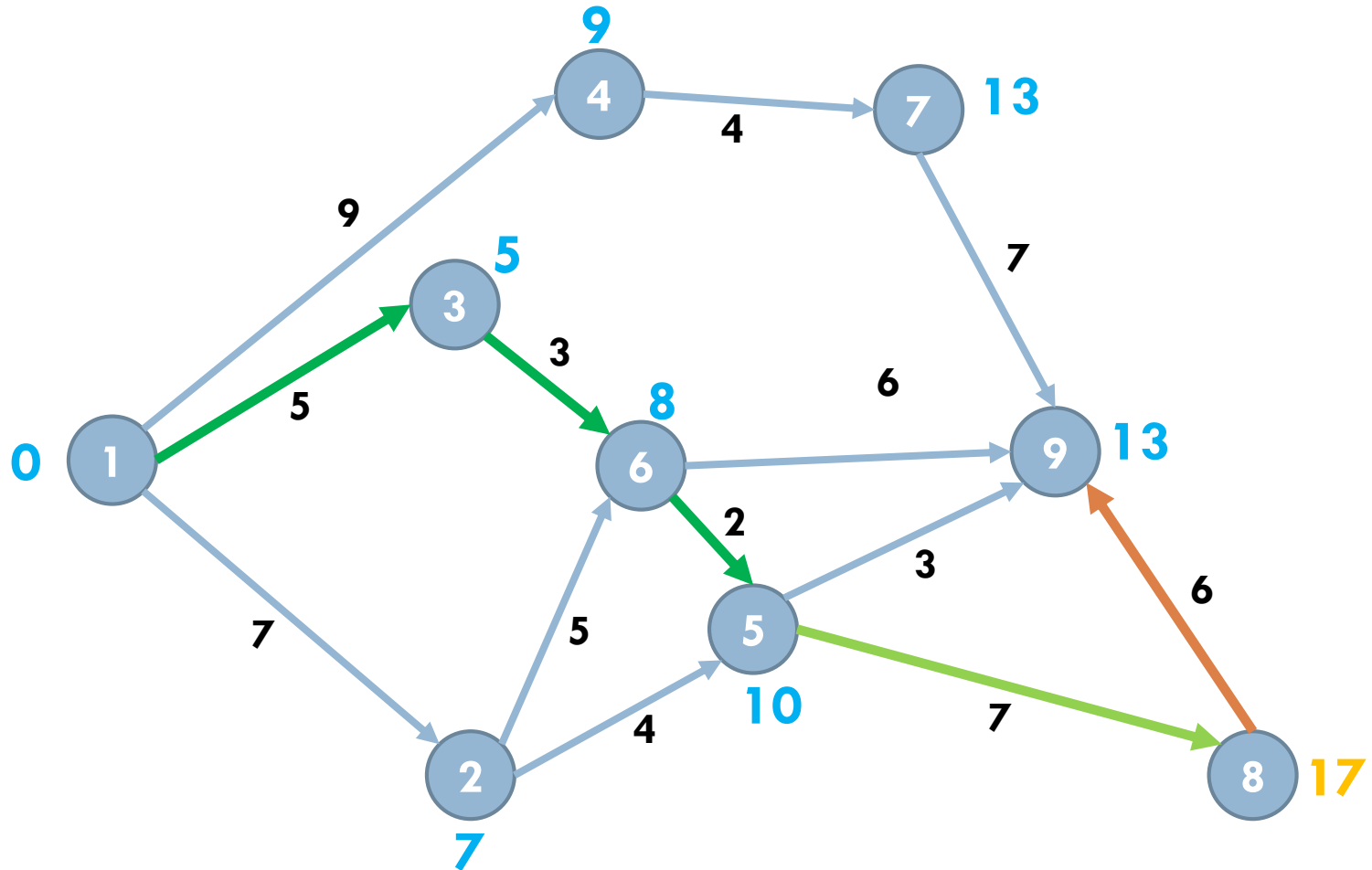
Najkraći put – Dijkstra



Najkraći put – Dijkstra



Najkraći put – Dijkstra



Najkraći put – Dijkstra (1)

```
long Graph::shortestPathDijkstra(int a,  
                                int typeImpementation){  
    long retVal = 0;  
    Node* ptr = start;  
    while (ptr != nullptr) {  
        ptr->status = 1;  
        ptr->prev = nullptr;  
        ptr->distance  
            = numeric_limits<double>::max();  
        ptr = ptr->next;  
    }  
    ptr = findNode(a);  
    if (ptr == nullptr) return 0;
```

// 2. deo koda

Najkraći put – Dijkstra (2)

// 1. deo koda

```
PriorityQueue *priorityQueue = nullptr;
if (typeImplementation == 1)
    priorityQueue = new PriorityQueueAsArray(nodeNum);
else if (typeImplementation == 2)
    priorityQueue = new PriorityQueueAsMinHeap(nodeNum);
ptr->distance = 0.0;
priorityQueue->insert(ptr);
ptr->status = 2;
while (!priorityQueue->isEmpty()) {
    ptr = priorityQueue->removeFirst();
    ptr->status = 3;
    retVal += 1;
}
```

// 3. deo koda

```
    }
    return retVal;
}
```

Najkraći put – Dijkstra (3)

// 2. deo koda

```
Edge* pot = ptr->adj;
while (pot != nullptr) {
    if (pot->dest->status == 1) {
        pot->dest->distance = ptr->distance + pot->weight;
        pot->dest->status = 2;
        pot->dest->prev = ptr;
        priorityQueue->insert(pot->dest);
    } else if (pot->dest->status == 2)
        if (pot->dest->distance > ptr->distance + pot->weight) {
            double newDistance = ptr->distance + pot->weight;
            pot->dest->prev = ptr;
            priorityQueue->update(pot->dest, newDistance);
            pot->dest->distance = newDistance;
        }
    pot = pot->link;
}
```

// 2. deo koda

Najkraći put – Dijkstra

Red sa prioritetom

- Operacije koje se pozivaju su:
- `priorityQueue->isEmpty()`
- `priorityQueue->removeFirst()` ;
- `priorityQueue->insert(pot->dest)` ;
- `priorityQueue->update(pot->dest, newDistance)` ;

Najkraći put – Dijkstra

Red sa prioritetom

operacija	uređeni niz
<code>isEmpty</code>	
<code>removeFirst</code>	
<code>insert</code>	
<code>update*</code>	
Efikasnost algoritma	

Najkraći put – Dijkstra

Red sa prioritetom

operacija	uređeni niz
<code>isEmpty</code>	$O(1)$
<code>removeFirst</code>	$O(n)$
<code>insert</code>	$O(n)$
<code>update*</code>	$O(\log n + n?)$
Efikasnost algoritma	$O((m+n+1) \cdot n) = O(m \cdot n)$

Najkraći put – Dijkstra

Red sa prioritetom

operacija	uređeni niz	okrenuti uređeni niz
isEmpty	$O(1)$	$O(1)$
removeFirst	$O(n)$	$O(1)$
insert	$O(n)$	$O(n)$
update*	$O(n^2)$	$O(n^2)$
Efikasnost algoritma	$O((m+n) \cdot n) = O(m \cdot n)$	$O((m+2) \cdot n) = O(m \cdot n)$

Najkraći put – Dijkstra

Red sa prioritetom

operacija	uređeni niz	okrenuti uređeni niz	gomila
<code>isEmpty</code>	$O(1)$	$O(1)$	
<code>removeFirst</code>	$O(n)$	$O(1)$	
<code>insert</code>	$O(n)$	$O(n)$	
<code>update*</code>	$O(n?)$	$O(n?)$	
Efikasnost algoritma	$O(m \cdot n)$	$O(m \cdot n)$	

Najkraći put – Dijkstra

Red sa prioritetom

operacija	uređeni niz	okrenuti uređeni niz	gomila
isEmpty	$O(1)$	$O(1)$	$O(1)$
removeFirst	$O(n)$	$O(1)$	$O(\log n)$
insert	$O(n)$	$O(n)$	$O(\log n)$
update*	$O(n?)$	$O(n?)$	$O(n + \log n)$
Efikasnost algoritma	$O(m \cdot n)$	$O(m \cdot n)$	$O(n \cdot (1 + \log n) + m \cdot (2 \cdot \log n + n)) = O(m \cdot n)$

Najkraći put – Dijkstra

Red sa prioritetom

operacija	uređeni niz	okrenuti uređeni niz	gomila	gomila + čvor sa indeksom
isEmpty	$O(1)$	$O(1)$	$O(1)$	$O(1)$
removeFirst	$O(n)$	$O(1)$	$O(\log n)$	$O(\log n)$
insert	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$
update*	$O(n?)$	$O(n?)$	$O(n + \log n)$	$O(\log n)$
Efikasnost algoritma	$O(m \cdot n)$	$O(m \cdot n)$	$O(m \cdot n)$	$O(n \cdot (1 + \log n) + 2 \cdot m \cdot \log n) = O(m \cdot \log n)$